

UNIT – 1: Introduction to Deep Learning

History of AI → ML → DL

Artificial Intelligence (AI - 1950s)

"What if machines could think like humans?"

So they started teaching computers with **rules**.

- Example: "If it has 4 wheels, it's a car."
- "If it has 2 wings, it's a plane"

But the problem was... the world is complicated

- Not all cars look the same.
- Some planes don't look like "normal planes."
So writing rules for everything became impossible.

Machine Learning (ML 1980s-2000s)

"Instead of writing rules, let the computer learn from examples"

- Example: Show the computer 1000 pictures of cats and dogs.
- It studies them and learns patterns: "Cats usually have smaller noses, dogs bark, dogs have longer ears."

Now, the computer could **predict** new things without strict rules.

But there was still a problem... humans had to **decide which features to show**.

- Like: "Look at the ears, look at the nose, look at the fur."

Deep Learning (DL - 2010)

Computers got faster, internet gave us **huge data**, and scientists built **neural networks with many layers** (deep networks).

Now, the computer didn't need humans to pick features.

- It looks at raw data (pixels in an image) and **figures out everything itself**
- First layer: finds edges.
- Next layers: find eyes, nose.
- Final layer: says "It's a cat"

This is why today we have:

- Self-driving cars
- ChatGPT
- Generative AI

AI = Baby : Needs parents to set rules.

ML = Kid : Learns from examples with some teacher guidance.

DL = Adult : Learns on its own, discovers patterns, becomes independent.

Difference between Machine Learning & Deep Learning

Feature	Machine Learning (ML)	Deep Learning (DL)
Definition	Teaches computers to learn from data using algorithms.	A special type of ML using neural networks with many layers.
Feature Extraction	Needs human help to select features (e.g., ear size, fur color in cat/dog detection).	Learns features automatically from raw data (edges → shapes → whole object).
Data Requirement	Works well with small to medium datasets .	Needs huge datasets (millions of images/text samples).
Computation Power	Runs on normal CPUs.	Needs high computing power (GPUs, TPUs).
Training Time	Relatively fast (minutes to hours).	Slow (hours to weeks for large models).
Examples	Spam filter, predicting house prices, recommendation systems.	Self-driving cars, Face Recognition, ChatGPT, Image Generators.
Analogy	Like a school student who needs a teacher to explain what to look at.	Like a grown-up adult brain that figures things out on its own.

Biological vs. Artificial Neural Networks

Feature	Biological Neural Networks (Human Brain)	Artificial Neural Networks (Computer Brain)
Basic Unit	Neuron (nerve cell) with dendrites & axon.	Artificial neuron (perceptron) – a simple math function.
How it Works	Neurons receive signals (electrical impulses), process them, and pass them on.	Neurons take numbers as input, apply weights & activation, and give output.
Connections	Trillions of synapses connect ~86 billion neurons.	Layers of nodes connected by weights (parameters).
Signal Transmission	Uses electro-chemical signals in the brain.	Uses mathematical operations (dot products, activations).
Learning Method	Learns by adjusting synapse strengths (neuroplasticity).	Learns by adjusting weights using backpropagation & gradient descent .
Speed	Slower (milliseconds per signal), but massively parallel.	Faster (nanoseconds per operation), but depends on computing power.
Example	Child recognizing a cat after seeing it a few times.	Neural net trained on millions of cat pictures to recognize cats.

Applications of Deep Learning

- Face recognition and biometric systems
- Speech recognition
- Machine translation
- Self-driving cars and traffic analysis
- Medical image analysis and disease detection
- Recommendation systems (movies, shopping, music)
- Fraud detection in banking and finance
- Document and handwriting recognition
- Industrial robotics and automation
- Generative AI (text, images, music creation)

Basics of Python Frameworks

1. TensorFlow

- Created by Google.
- Very powerful, used in both research and industry.
- Good for building large, complex deep learning models.

2. Keras

- A simpler, user-friendly interface that usually runs on top of TensorFlow.
- Lets you build deep learning models in just a few lines of code.
- Best for beginners.

3. PyTorch

- Created by Facebook.
- Very flexible and popular in research.
- Easy to experiment with new model ideas.

Unit 2: Fundamentals of Neural Networks

Perceptron model (single-layer)

A perceptron is the **simplest model of a neural network**. It is just **one neuron** that makes a decision based on inputs.

How does it work?

1. **Inputs come in** (like yes/no or numbers).
 - Example: Is it sunny? (1 or 0), Do you have free time? (1 or 0).
2. Each input has a **weight** (importance).
 - Maybe "sunny" is more important than "free time."
3. The perceptron **adds up inputs × weights**.
4. It compares the result with a **threshold**.
 - If result > threshold → Output = 1 (Yes).
 - If result ≤ threshold → Output = 0 (No).

Que : Should I go for a walk?

- Input 1: Weather is good (1 if yes, 0 if no).
- Input 2: I am free (1 if yes, 0 if no).
- If both are true and their sum is greater than the threshold → Perceptron says "Yes, go for a walk."
- Otherwise → "No."

Limitation

- A single perceptron can only handle **simple decisions** (like AND, OR).
- It cannot solve more complex problems (like XOR) because it has no hidden layers.

Multi-Layer Perceptron (MLP)

A **Multi-Layer Perceptron** is just a **bigger version of a single perceptron**.

Instead of only one layer of decision-making, it has **many layers** stacked together:

- **Input Layer** → where data comes in.
- **Hidden Layers** → where learning and pattern recognition happen.
- **Output Layer** → gives the final result.

How does it work?

1. Inputs are passed into the network.
2. Each hidden layer transforms the inputs step by step.
3. By the end, the output layer gives the answer.

Imagine recognizing a handwritten digit “5”.

- **Input Layer:** Takes the raw pixels of the image.
- **Hidden Layer 1:** Finds small patterns like edges and curves.
- **Hidden Layer 2:** Combines edges into shapes like circles and lines.
- **Hidden Layer 3:** Recognizes the overall structure as a digit.
- **Output Layer:** Decides “This is 5.”

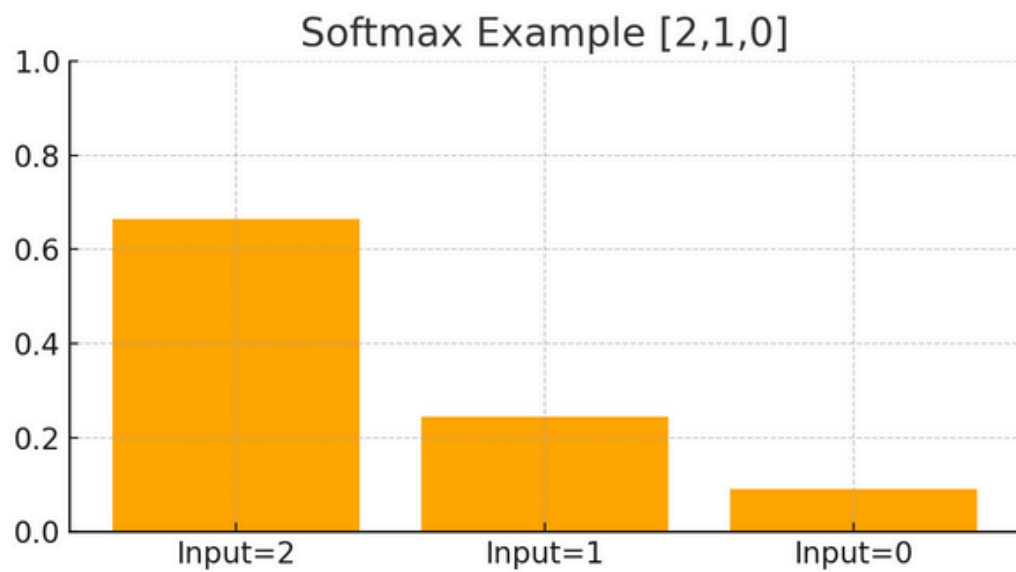
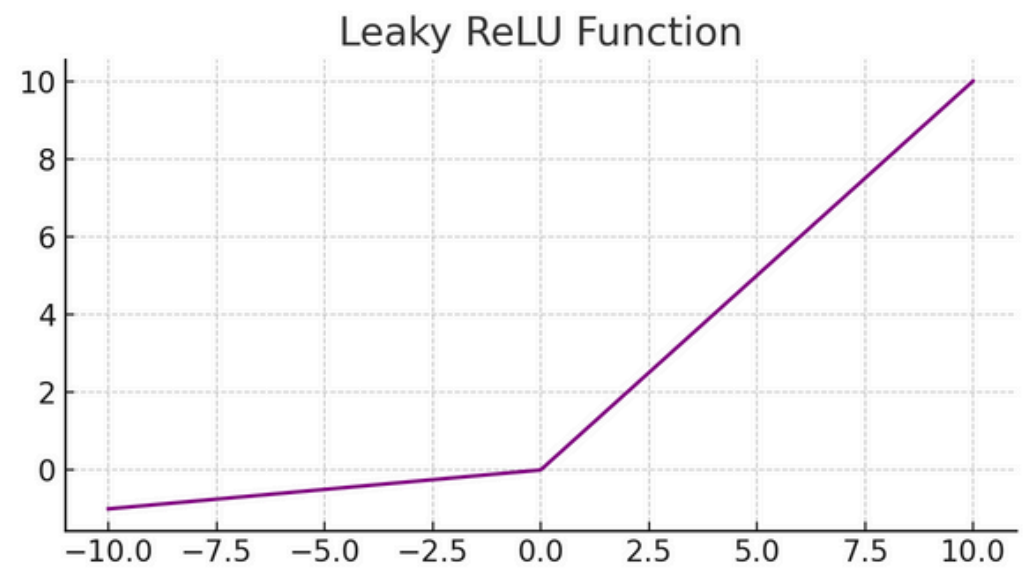
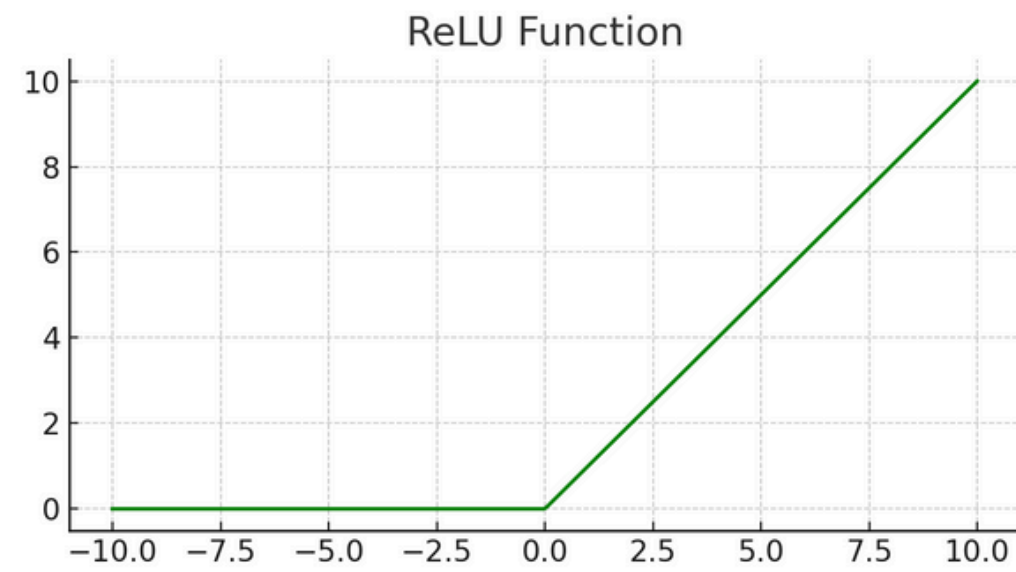
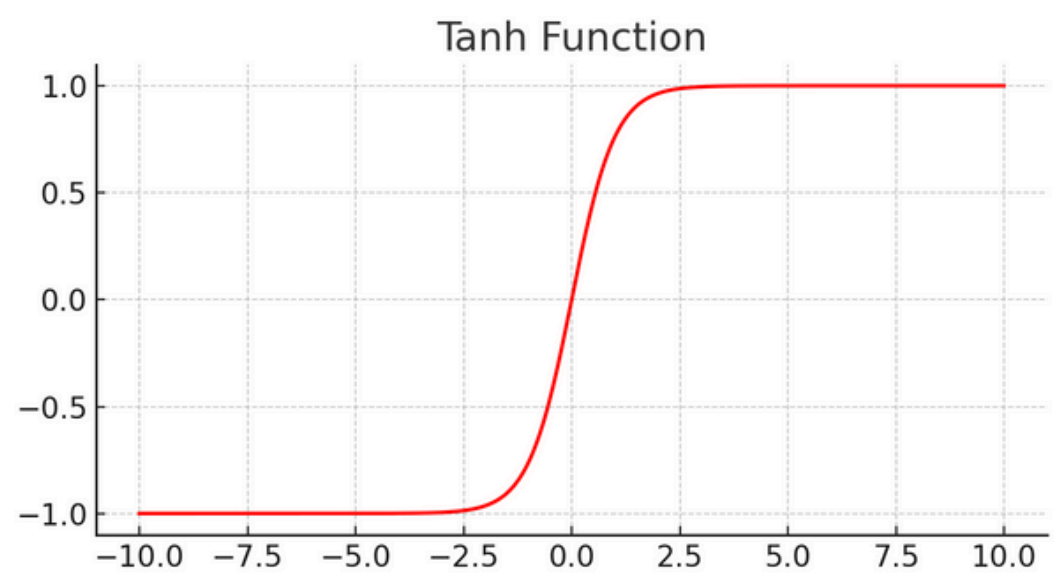
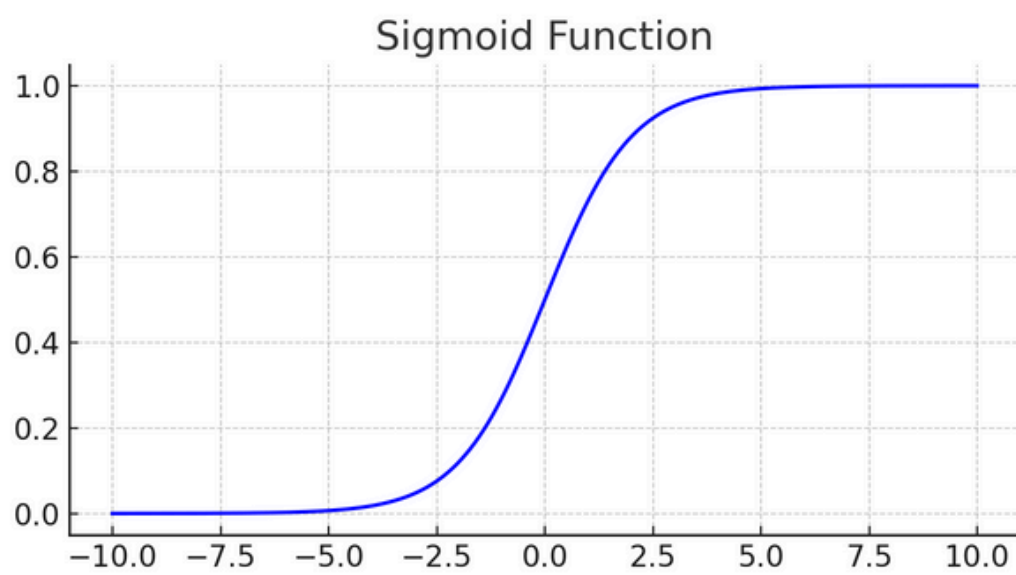
Limitation

- Needs a lot of data to train.
- Takes longer time and more computing power.

Activation Functions (Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax)

Activation functions decide whether a neuron should “fire” (activate) or not. They add **non-linearity**, which helps networks learn complex things.

Function	Formula	Output Range	Example Output
Sigmoid	$1/(1+e^{(-x)})$	(0,1)	$x=2 \rightarrow 0.88$
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	(-1,1)	$x=2 \rightarrow 0.96$
ReLU	$\max(0,x)$	$[0,\infty)$	$x=-3 \rightarrow 0, x=5 \rightarrow 5$
Leaky ReLU	$x \ (x>0), 0.01x \ (x\leq 0)$	$(-\infty,\infty)$	$x=-4 \rightarrow -0.04$
Softmax	$e^{x_i} / \sum e^{x_j}$	(0,1), sum=1	$[2,1,0] \rightarrow [0.66,0.24,0.09]$



Gradient Descent & Backpropagation

Gradient Descent

It's a way to find the **best values of weights** in a neural network so that predictions are accurate.

- Imagine a **mountain landscape** with valleys.
- You drop a ball on the mountain, and it **rolls downhill**.
- The lowest valley represents the **minimum error (loss)**.

So, gradient descent is like the ball rolling down the slope, step by step, until it finds the lowest point.

Backpropagation

It's how the network **adjusts weights** after making a mistake.

Step by step:

1. Input flows **forward** through the network → gives prediction.
2. Error (loss) is calculated.
 - Error = difference between prediction and correct answer.
3. Backpropagation sends this error **backward** through the network.
 - Each neuron checks: "How much did I contribute to this error?"

4. Weights are updated slightly using **gradient descent**.
 - Next time, the prediction improves.

Loss Functions

1. Mean Squared Error (MSE)

Formula:

$$\text{MSE} = (1/n) * \sum (y_i - y_p)^2$$

- y_i = actual value
- y_p = predicted value

Where used: Regression (predicting continuous values).

Example:

Actual = [3, 5], Predicted = [2, 6]

$$\text{MSE} = ((3-2)^2 + (5-6)^2) / 2$$

$$= (1 + 1) / 2$$

$$= 1$$

2. Cross-Entropy Loss

Binary Classification Formula:

$$\text{Loss} = - [y * \log(y_p) + (1-y) * \log(1-y_p)]$$

- y = true label (0 or 1)
- y_p = predicted probability

Example:

True = 1, Predicted probability = 0.9

$$\text{Loss} = -(1 * \log(0.9) + 0 * \log(0.1))$$

$$= -\log(0.9)$$

$$\approx 0.10$$

If predicted probability = 0.1 (very wrong):

$$\text{Loss} = -\log(0.1)$$

$$= 2.30$$

3. Hinge Loss

Formula:

$$\text{Loss} = \max(0, 1 - y * y_p)$$

- $y \in \{-1, +1\}$ (true label)
- y_p = predicted score (not probability, raw output)

Example:

True = +1, Predicted = 0.8

$$\text{Loss} = \max(0, 1 - (1 * 0.8))$$

$$= \max(0, 0.2)$$

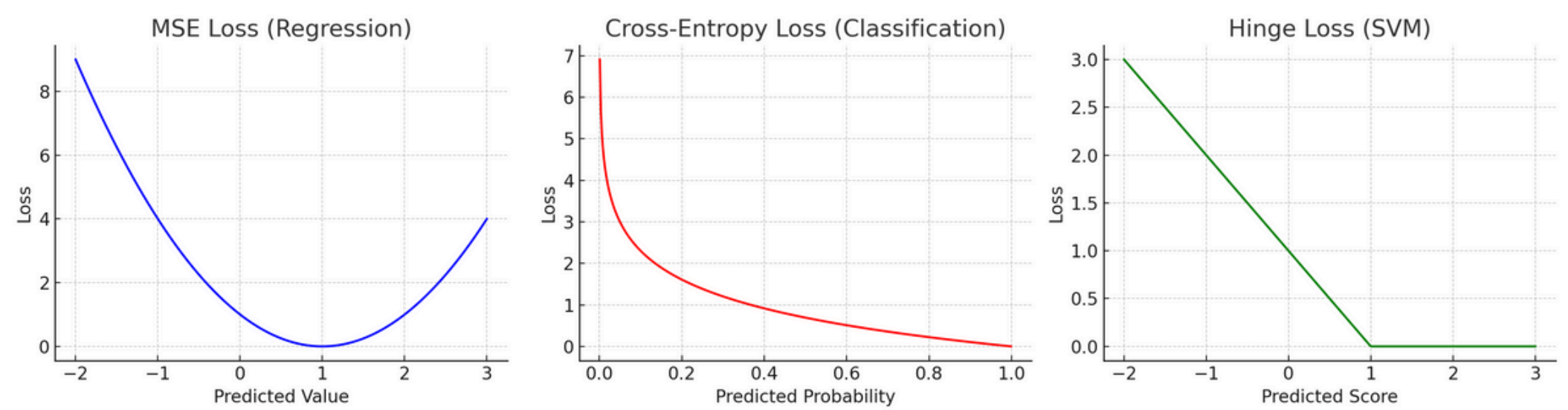
$$= 0.2$$

True = +1, Predicted = 1.5

$$\text{Loss} = \max(0, 1 - (1 * 1.5))$$

= max(0, -0.5)

= 0



MSE : parabolic curve, error grows as prediction moves away from true value.

Cross-Entropy : very steep penalty when predicted probability is wrong.

Hinge : linear margin; loss is 0 only when prediction is confidently correct.

Optimization algorithms (SGD, MiniBatch SGD, RMSProp, Adam)

Optimizer	Intuition	Pros	Cons	Common Use
SGD (Stochastic Gradient Descent)	Learns step by step, adjusting weights immediately after each training sample	Simple, easy to implement- Works well on large datasets- Can escape local minima due to noise	Noisy updates- Convergence is slow- May oscillate around minimum	Online learning, very large datasets
Mini-Batch SGD	Learns from small groups of data instead of one or all	Faster than batch GD- Smoother than pure SGD- Efficient use of hardware (vectorization, GPUs)	Still noisy- Performance depends on batch size	Deep learning training (default choice)
RMSProp (Root Mean Square Propagation)	Adjusts step size per parameter; slows down when gradient is large, speeds up when small	Handles non-stationary objectives well- Good for RNNs and deep networks- Adapts learning rates	May overshoot- Needs tuning of β (decay rate)	Training deep RNNs, non-convex problems
Adam (Adaptive Moment Estimation)	Combines momentum (m) + RMSProp (v) $\rightarrow$ faster and adaptive	Most widely used- Fast convergence	Slightly more memory use- Can sometimes overfit- Not always best for very large-scale problems	Default optimizer in TensorFlow, PyTorch; NLP models, CV models

SGD : Moves step by step, zig-zagging toward the bottom.

Mini-Batch SGD : Smoother than SGD but still a bit noisy.

RMSProp : Adjusts step size; descent is smoother and more controlled.

Adam : Fastest and smoothest convergence; combines momentum + adaptive learning rate.

Unit 3: Convolutional Neural Networks

Basics of Convolutional Neural Networks (CNNs)

A **Convolutional Neural Network (CNN)** is a special type of neural network designed to work with **images** (and sometimes videos, audio, etc.). It helps computers **see patterns in pictures** — just like our eyes and brain.

How does it work?

1. **Convolution (Feature Detection):**
 - Think of it as sliding a small “window” (filter) over the image.
 - Each filter looks for something specific: edges, corners, colors, textures.
 - The output is called a **feature map** (like a highlight of where that pattern appears).
2. **Pooling (Downsampling):**
 - After detecting features, the image is made **smaller** but important parts are kept.
 - Example: **Max pooling** takes the strongest signal (brightest pixel) in each block.
 - This makes CNNs faster and less sensitive to small changes.
3. **Fully Connected Layers:**
 - At the end, the CNN takes all the features and tries to make a decision.
 - Example: “This image is 90% dog, 10% cat.”

Why CNNs are powerful?

- They **automatically learn** what features matter (you don’t tell them “look at the ear” — they figure it out).
- They are good at handling big image data.
- They mimic how our **visual system** works (eyes detect edges → brain combines them → recognizes object).

Imagine looking at a picture of a car.

- First, your eyes notice edges and shapes (wheels, windows).
- Then, your brain puts them together to say “Car.”

CNNs work the same way: filters detect parts → combine → final classification.

Convolution Operation, Filters, Feature Maps

1. Convolution Operation

- Convolution is like **sliding a small window (matrix) across the image** and doing math at each step.
- At each position, we multiply the numbers in the window with the numbers in the image and add them up.
- The result tells us **how strongly that pattern exists in that part of the image**.

Example:

Imagine you have a black-and-white image of a cat (numbers represent pixel brightness).

If you slide a filter that looks for **vertical edges**, the output highlights the whiskers and body edges.

2. Filters (Kernels)

- A **filter** (or kernel) is just a tiny matrix, like 3x3 or 5x5.
- Each filter is designed to detect something specific:
 - Vertical edges
 - Horizontal edges
 - Corners
 - Textures

Example:

- Vertical edge filter = $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$
- If you pass it over an image of a zebra, it will highlight the stripes!

3. Feature Maps

- The **feature map** is the output after applying the filter.
- It shows **where in the image** that feature appears.
- Bright spots in the feature map mean “this filter found something important here.”

Example:

- If you apply a filter for circles on a coin image → feature map lights up where the coins are.
- If you apply an edge filter → feature map shows outlines of objects.

Convolution: The process of scanning the image with a filter.

Filter: A tiny pattern detector (matrix).

Feature Map: The result, showing where that pattern exists in the image.

Pooling Layers (Downsampling)

Pooling is like **shrinking the image/feature map** while keeping only the most important details. It makes the network faster and prevents it from memorizing tiny details (overfitting).

Types of Pooling

(a) Max Pooling

- Takes the **maximum value** from a small block (like 2×2).
- Keeps the **strongest signal**.
- Example: If a block has values [2, 5, 1, 3], max pooling → 5.

(b) Average Pooling

- Takes the **average value** of the block.
- Smooths out the features.
- Example: [2, 5, 1, 3] → average = 2.75.

(c) Global Pooling

- Instead of a small block, it shrinks the **entire feature map** into a single number.
- Used at the end of CNNs before classification.

Why Pooling is Needed?

- **Reduces size** → Makes CNN faster.
- **Keeps important info** → Small details are ignored.
- **Makes CNN stable** → Small shifts (like moving a cat's ear a little) won't confuse the model.

Imagine showing a photo to your friend but in **low resolution**:

- They can't see every hair, but they still know it's a **dog**.
- That's what pooling does → reduces details but keeps the "big picture."

Dropout, Batch Normalization, Regularization Techniques

Dropout

What it is:

- During training, **random neurons are "turned off"** (ignored).
- Example: If a layer has 100 neurons, Dropout might temporarily use only 70 each time.

Why it helps:

- Prevents the network from depending too much on specific neurons.
- Makes the model more **robust** and avoids **overfitting** (memorizing training data).

Real-life analogy:

Imagine a cricket team practicing, but sometimes their best batsman sits out. The rest of the team learns to play better — not just depending on one player.

Batch Normalization (BN)

What it is:

- Adjusts (normalizes) the output of each layer so the data going forward has a **stable distribution** (mean ≈ 0 , variance ≈ 1).
- Done **during training** for each mini-batch of data.

Why it helps:

- Speeds up training (faster convergence).
- Prevents values from becoming too large/small (gradient explosion/vanishing).
- Acts like a light form of regularization.

Example:

Think of it like **checking students’ mood in a class** — if some are too hyper and some are sleepy, the teacher balances everyone so learning goes smoothly.

Regularization Techniques

Regularization = techniques to **reduce overfitting** (when the model performs well on training but poorly on new data).

Common Types:

- **L1 Regularization (Lasso):**
 - Adds penalty = sum of absolute weights.
 - Makes many weights exactly **zero** → feature selection.

Formula: Original Loss + $\lambda * \sum |w|$

- **L2 Regularization (Ridge):**
 - Adds penalty = sum of squared weights.
 - Keeps weights small, but rarely zero.

Formula: Original Loss + $\lambda * \sum (w^2)$

Famous CNN Architectures

Architecture	Year	Main Idea	Famous For
LeNet-5	1998	First CNN	Digit recognition
AlexNet	2012	Deep CNN + GPU + ReLU	ImageNet breakthrough
VGGNet	2014	Many 3×3 filters	Depth & simplicity
GoogLeNet	2014	Inception module	Efficient + parallel filters
ResNet	2015	Skip connections	Ultra-deep networks
YOLO	2016	One-shot detection	Real-time object detection
Faster R-CNN	2015	Region proposals + CNN	Accurate object detection

Unit 4: Recurrent Neural Networks (RNNs)

Basics of RNN (Recurrent Neural Network)

Normal Neural Network Problem

- A normal neural net takes **fixed inputs** and gives **fixed outputs**.
- But it **doesn't remember** what came before.
- Example: If you give the word *"love"*, it cannot know what came before (*"I"*) to predict correctly.

What RNN Does

- RNN is designed for **sequences** (data that comes in order).
- It has a **memory (hidden state)** that stores past information and passes it to the next step.

Think of it like **reading a story word by word** — you always remember the earlier words to understand the current word.

How It Works (Step by Step)

- Input at time $t \rightarrow$ processed by the network.
- Hidden state (memory) from time $t-1$ is also used.
- Output + updated hidden state are produced.
- Then the process repeats for the next time step.

Formula:

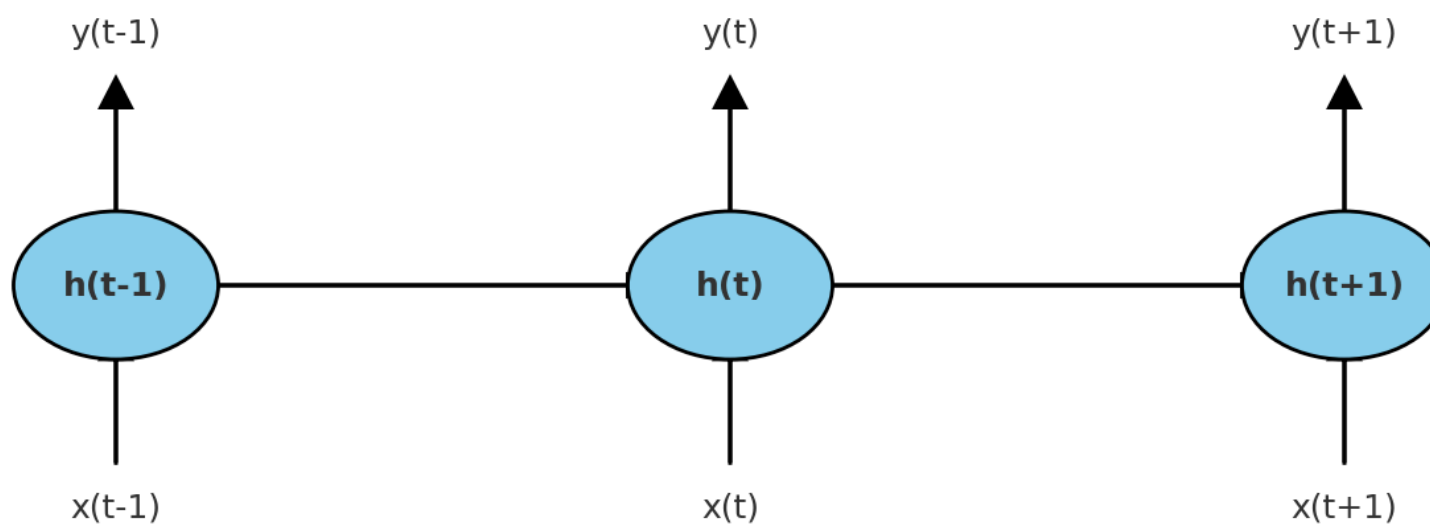
$$h_t = f(W * x_t + U * h_{t-1})$$

$$y_t = g(V * h_t)$$

- h_t = hidden state at time t
- x_t = input at time t
- y_t = output
- W, U, V = weights
- f, g = activation functions

RNN = a neural network with **memory**, used for sequential data like text, speech, and time-series.

Unfolded Recurrent Neural Network (RNN) through time



Sequential Data & Time-Series Modeling

Sequential data = data where **order matters**.

- In normal datasets, you can shuffle rows and nothing changes (e.g., a table of student marks).
- But in sequential data, **past values affect future values**.

Examples:

- A sentence $\rightarrow$ words must be in order (*"I love you"* $\neq$ *"You love I"*).
- Stock prices $\rightarrow$ today's value depends on yesterday's.
- Music $\rightarrow$ the next note depends on the earlier ones.

Key idea: We cannot treat each data point separately; we must consider the **sequence**.

Time-series is a **special type of sequential data** where values are recorded over **time**.

Examples:

- Temperature recorded every hour.
- Daily sales of a shop.
- ECG signals from the human heart.

Time-series = “sequences indexed by time.”

Why Do We Model Sequences?

We model sequences to **predict, analyze, or generate** future values/events.

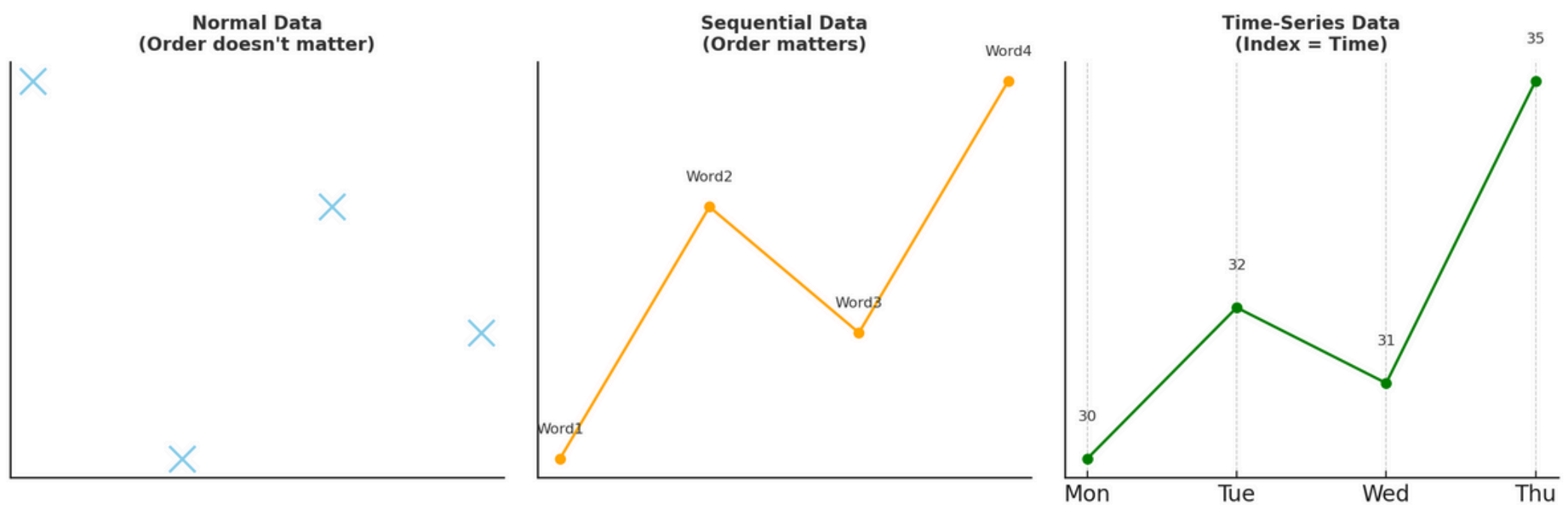
Examples:

- **Forecasting**: Predict next week’s weather from past temperatures.
- **Language modeling**: Predict the next word in a sentence.

Simple Real-Life Example

Imagine you are watching a cricket match:

- Just looking at the current score (**e.g., 120/3**) isn’t enough.
- You need the **sequence of overs and runs** to guess how the game is progressing.
- This is exactly why sequential modeling is important — context from the **past** improves understanding and prediction.



Vanishing & Exploding Gradients problem

Why Does This Problem Happen?

When we train RNNs, we use **backpropagation through time (BPTT)**.

- Gradients (learning signals) are multiplied many times as they travel back through each time step.
- Depending on the values:
 - If weights < 1 → gradient **shrinks** → **vanishing gradient**.
 - If weights > 1 → gradient **grows uncontrollably** → **exploding gradient**.

Vanishing Gradient

- The gradient becomes **tiny** as it moves backward.
- Network **forgets long-term dependencies**.
- Example:
 - Sentence: “I was born in Paris ... I speak ____”

- RNN cannot connect “Paris” to “French” because the signal vanished.

Old information is **lost**.

Exploding Gradient

- The gradient becomes **huge** as it moves backward.
- Causes **unstable training**, weights jump around, loss fluctuates.
- Example: Model’s output changes randomly instead of learning properly.

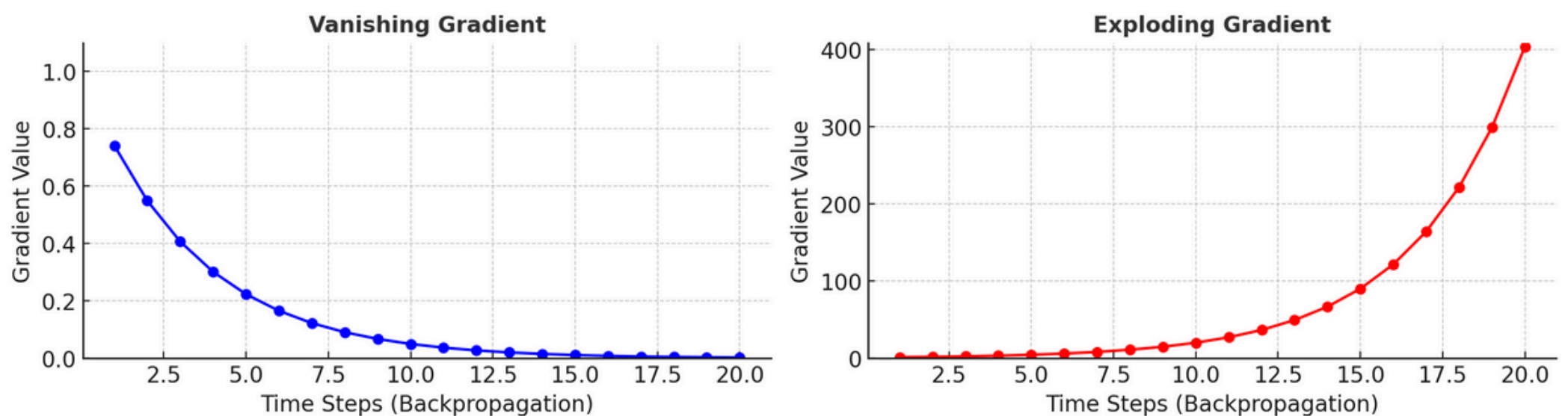
Training becomes **unstable**.

Vanishing gradient → network forgets old information.

Exploding gradient → network becomes unstable.

Solution → Use LSTM.

Vanishing vs Exploding Gradients in RNNs



Vanishing Gradient : The gradient value **shrinks** as we go back in time steps → old information gets lost.

Exploding Gradient : The gradient value **blows up** as we go back in time steps → training becomes unstable.

Long Short-Term Memory (LSTM) Networks

- RNNs forget old information (vanishing gradient problem).
- Example: “I grew up in Paris ... I speak ____” → Vanilla RNN forgets “Paris.”
- **LSTM solves this by adding a “memory cell”** that can keep important information for long time.

Think of LSTM as an **RNN with memory + gates** to control what to remember and what to forget.

The Key Idea of LSTM

Inside each LSTM cell, there are **three gates + one memory cell**:

1. **Forget Gate (forget old info):**
Decides what past info to throw away.
Example: Old cricket scores from 10 overs ago may not matter.
2. **Input Gate (accept new info):**
Decides what new info to add.
Example: Current over’s runs are important → add them.
3. **Output Gate (decide what to show):**
Decides what part of memory to use as output.
Example: Predicting next score based on current + past.
4. **Cell State (memory box):**
Long-term memory → information highway running through the network.

GRU (Gated Recurrent Unit)

Why GRU?

- LSTM is powerful but **a bit complex** (3 gates + memory cell).
- GRU is a **simpler version of LSTM** → fewer gates, faster training, but still solves vanishing gradient.
- Think of GRU as **“LSTM Lite”**.

GRU Structure

GRU has only **2 gates** (instead of 3 in LSTM):

1. **Update Gate (z_t):**
 - Decides how much of the past to keep and how much new info to add.
 - Works like **forget + input gate combined**.
2. **Reset Gate (r_t):**
 - Decides how much of the past to forget when mixing new input.

👉 Because of fewer gates, GRUs are **faster** and need **less data** to train compared to LSTMs.

Feature	LSTM	GRU
Gates	3 (Forget, Input, Output)	2 (Update, Reset)
Memory Cell	Yes	No (uses hidden state only)
Speed	Slower	Faster
Performance	Slightly better for complex tasks	Similar, often faster for small datasets

Applications of RNN, LSTM, and GRU

1. Text Generation

- The model learns the style of writing and generates new text.
- **Example:**
 - Train on Shakespeare’s plays → generate new lines in Shakespeare style.
 - Train on your WhatsApp chats → generate messages like you.

2. Speech Recognition

- Converts spoken words into text.
- **Example:**
 - Siri, Alexa, Google Assistant converting voice → commands.
 - Call centers using speech-to-text.

3. Sentiment Analysis

- Understands emotions in text.
- **Example:**
 - Reviews: “The movie was awesome!” → Positive.
 - Tweets: “This service is terrible!” → Negative.

4. Machine Translation

- Translate text from one language to another.
- **Example:**
 - English: “I am going to school.”
 - Hindi: “मैं स्कूल जा रहा हूँ।”

5. Time-Series Forecasting

- Predict future values from past sequences.
- **Example:**
 - Stock price prediction.
 - Weather forecasting (temperature, rainfall).

- Sales prediction for a company.

Unit 5: Advanced Deep Learning Architectures

Autoencoders

An **autoencoder** is like a **data compressor + decompressor**.

It takes input data → compresses it to a smaller form (encoding) → then reconstructs it back (decoding).

Structure of Autoencoder

1. **Encoder** → Shrinks input into a smaller hidden form (latent space).
2. **Latent Space (Code)** → The compressed knowledge.
3. **Decoder** → Reconstructs the original input from compressed code.

Think of it like **zipping and unzipping a file**.

Types of Autoencoders

1. Basic Autoencoder

- Learns to copy input → output.
- Example:
 - Input: Handwritten “7” → Encoder compresses → Decoder reconstructs → Output: “7”.
- Use case: **Dimensionality reduction** (similar to PCA), feature extraction.

2. Denoising Autoencoder

- Learns to remove noise from data.
- Example:
 - Input: A noisy photo (blur, scratches).
 - Output: A cleaned version.
- Use case: **Image enhancement, removing background noise in audio**.

3. Variational Autoencoder (VAE)

- Special type → doesn't just copy data but **generates new data** similar to training data.
- Example:
 - Train VAE on faces → It can generate **new human faces** that never existed.
- Use case: **Generative tasks (image synthesis, anomaly detection, drug design)**.

Basic Autoencoder → Like compressing a movie into a zip file and unzipping it back.

Denoising Autoencoder → Like Photoshop's “Auto Fix” that removes blur/noise.

VAE → Like an artist who learns a style and then paints **new artworks in that style**.

Generative Adversarial Networks (GANs)

A **GAN** has **two neural networks fighting each other**:

1. **Generator (G)** → Creates fake data (tries to fool).
2. **Discriminator (D)** → Judges if data is real or fake.

Together, they improve until the generator becomes so good that it produces **realistic data**.

Structure of a GAN

1. **Generator**
 - Input: Random noise

- Output: Fake but realistic-looking data (like fake images)
- Goal: Fool the discriminator

2. **Discriminator**

- Input: Both real data + fake data from generator
- Output: Probability (real or fake)
- Goal: Catch the fake

Training Process

- **Step 1:** Generator makes fake images
- **Step 2:** Discriminator checks → “Real or Fake?”
- **Step 3:** If Discriminator catches → Generator improves
- **Step 4:** If Generator fools Discriminator → Discriminator improves
- **Repeat** until Generator makes data almost identical to real

Real-Life Example

- Think of a **student (Generator)** writing fake currency notes
- A **police officer (Discriminator)** inspects them
- If the officer catches → student improves next time
- If the student fools → officer trains harder
- Over time, the student becomes a **master at making realistic notes**

Basics of Transformers & Attention Mechanism

Problem Before Transformers

RNNs and LSTMs worked for sequences (like text), but:

- They read **one word at a time** → too slow.
- They **forget long context** → bad at long sentences.
- Hard to parallelize → training took ages.

That’s where **Transformers** came in (2017, “*Attention is All You Need*”).

Core Idea: Attention Mechanism

Imagine you’re reading this sentence:

“The dog chased the ball because **it** was rolling fast.”

Question: What does “**it**” refer to? → **the ball**.

- **Attention** helps the model focus on the **important word (ball)** instead of treating all words equally.
- It assigns **weights** (importance scores) to words.

So, when the model sees “it”, attention tells it: “Look at *ball*, not *dog*.”

Transformer Architecture

1. **Input Embedding** – Words converted into vectors.
2. **Positional Encoding** – Since order matters (cat vs act), position info is added.
3. **Encoder** – Reads the input with **multi-head attention** + feedforward layers.
4. **Decoder** – Uses its own attention + encoder info to generate output (e.g translation).
5. **Output** – Predicted text sequence.

Multi-Head Attention

- Instead of looking at just one relation, the model looks at **different angles simultaneously**.
- Example:
 - One head might focus on **subject-verb relation**.
 - Another on **object-adjective relation**.
- Then all heads combine for richer understanding.

Reinforcement Learning & Deep Q-Learning

Reinforcement Learning (RL) – Basics

Think of it like **training a pet dog**.

- You give the dog a **command** (action).
- If it does well → you give a **reward** (treat).
- If it does wrong → no treat or punishment.
- Over time, the dog **learns the best actions** to maximize rewards.

RL System has 3 main parts:

1. **Agent** → Learner/decision-maker (the dog, or the AI model).
2. **Environment** → World where the agent acts (your home, or a video game).
3. **Rewards** → Feedback signal (good = +1, bad = -1).

Cycle: Agent takes action → Environment responds → Agent learns → Repeats.

Deep Q-Learning (DQN) – Extension of RL

- In RL, agent often uses a **Q-table** (matrix of states & actions).
- But for **large/complex environments** (like games, robots, stock markets), Q-table becomes too big.

So we replace the Q-table with a **Deep Neural Network** → this is **Deep Q-Learning**.

DQN Working (in simple steps):

1. **Input:** Current state (e.g., game screen).
2. **Neural Network:** Predicts Q-values for all possible actions.
3. **Agent chooses action:** Best Q-value = action taken.
4. **Reward received** from environment.
5. **Network updated** using backpropagation.
6. Repeat → Agent learns optimal strategy.

Reinforcement Learning = Learning by **trial and reward**.

Deep Q-Learning = RL **powered by Deep Neural Networks** for large/complex problems.